

Yisus es mi pastor, nada me faltara



Contents

1	Data Structures	2
1.1	Priority Queue de minimos	2
1.2	Priority Queue de maximos	2
1.3	Tabla Aditiva - Partial Sum o Prefix Sum	2
2	Operaciones con bits en C++	2
2.1	Funciones utiles de C++	2
2.1.1	popcount	2
3	Grafos	3
3.1	Algoritmo Dijkstra	3
3.2	Algoritmo Bellman-Ford	3
3.3	Union Find	4
4	Manejo de Strings	4
4.1	Sub String	4
4.2	KMP	4
5	Trucos Varios	5
5.1	Next Permutation	5
5.2	Uso de Fixed	5
5.3	Equivante fixed con printf	6
5.4	iota	6

5.5	Conversion de unidades	6
5.6	Salidas varias	7
5.7	Aritmetica con Strings	7
5.8	Aritmetica con Strings segun deepseek	8
5.9	Problema M codigo	10
5.10	Matriz Viborita	11
5.11	Triangulo de Pascal	11
6	Geometry	12
6.1	Formulas Basicas	12
6.2	Convex Hull	12
6.3	Trigonometria	13
7	Math	14
7.1	Factores de un numero	14
7.2	Linear Sieve	14
7.3	Cribita normal	14
7.4	Criba vasito	14
7.5	Contar Divisores	15
7.6	Operaciones con matrices	15
7.7	Fast Exponentiation	16

1 Data Structures

1.1 Priority Queue de minimos

```

1 priority_queue<int, vector<int>, greater<int>> pq;
2 pq.push(x); // Inserta el elemento x
3 pq.top(); // Devuelve el menor elemento (minimo)
4 pq.pop(); // Elimina el menor elemento
5 pq.empty(); // Retorna true si esta vacia
6 pq.size(); // Numero de elementos

```

1.2 Priority Queue de maximos

```

1 priority_queue<int> pq;
2 pq.push(x); // Inserta el elemento x
3 pq.top(); // Devuelve el mayor elemento (maximo)
4 pq.pop(); // Elimina el mayor elemento
5 pq.empty(); // Retorna true si esta vacia
6 pq.size(); // Numero de elementos
7 //Coordenadas cilindricas entra fija

```

1.3 Tabla Aditiva - Partial Sum o Prefix Sum

```

1 vector<int> v = {1, 2, 3, 4, 5};
2 vector<int> prefix(v.size());
3 partial_sum(v.begin(), v.end(), prefix.begin() //posicion desde la que se
4 //construira);
5 for (int x : prefix)
6     cout << x << " ";
7 // Output: 1 3 6 10 15

```

2 Operaciones con bits en C++

2.1 Funciones utilices de C++

2.1.1 popcount

Para obtener el numero de bits encendidos en un entero

```

1 cout << __builtin_popcount(n) << '\n'; // siendo n un int

```

Version para numeros de 64bits (long long)

```

1 cout << __builtin_popcountll(n) << '\n'; // siendo n un long long

```

Hack bits

```

1 /*
2  * Bit Hacks
3  * Tcnicas tiles para trabajar con mscaras de bits.
4  * Muy utilizadas en programacin competitiva para problemas de
5  * conjuntos, DP con bitmasks, combinatoria, etc.
6  */
7 #include <bits/stdc++.h>
8 using namespace std;
9
10 using ull = unsigned long long;
11
12 /*
13  * next_bits_permutation(x)
14  * Devuelve la siguiente mscara con el mismo nmero de bits
15  * encendidos (1s) en orden lexicogrfico.
16  * til para generar todas las combinaciones de k elementos como
17  * mscaras de bits.
18  *
19  * Ejemplo:
20  * x = 0b00111 (representa una combinacin de 3 elementos)
21  * next_bits_permutation(x) 0b01011, luego 0b01101, etc.
22  */
23 ull next_bits_permutation(ull x) {
24     ull c = __builtin_ctzll(x); // Cuenta los ceros a la derecha
25     // de x
26     ull r = x + (1ULL << c); // Aade 1 al bit menos
27     // significativo que est en 1
28     return (r ^ x) >> (c + 2) | r; // Calcula la siguiente
29     // permutacin
30 }
31
32 /*
33  * subsets(s)
34  * Itera sobre todos los subconjuntos propios (no incluye s) de un
35  * conjunto dado s.
36  * El conjunto s se representa como una mscara de bits.
37  *
38  * Ejemplo:
39  * s = 0b1011 (conjunto {0,1,3})
40  * Se recorrern los subconjuntos: 1010, 1001, 1000, 0011, 0010, 0001
41  */
42 void subsets(ull s) {

```

```

37     for (ull x = s; x;) {
38         --x &= s;
39         // Aqu puedes trabajar con cada subconjunto x
40         // Por ejemplo, imprimirlo:
41         // cout << bitset<4>(x) << '\n';
42     }
43 }

```

3 Grafos

3.1 Algoritmo Dijkstra

```

1  /*
2  * Dijkstra's Algorithm
3  * Encuentra las distancias m nimas desde un nodo origen en un grafo
4  * ponderado sin pesos negativos.
5  * Complejidad:  $O((V + E) \log V)$ 
6  */
7  #include <bits/stdc++.h>
8  using namespace std;
9
10 const int INF = 1e9;
11 using pii = pair<int, int>; // {dist, node}
12
13 vector<int> dijkstra(int n, int src, const vector<vector<pii>>& adj) {
14     vector<int> dist(n, INF);
15     priority_queue<pii, vector<pii>, greater<>> pq;
16     dist[src] = 0;
17     pq.emplace(0, src);
18
19     while (!pq.empty()) {
20         auto [d, u] = pq.top(); pq.pop();
21         if (d > dist[u]) continue;
22         for (auto [v, w] : adj[u]) {
23             if (dist[v] > dist[u] + w) {
24                 dist[v] = dist[u] + w;
25                 pq.emplace(dist[v], v);
26             }
27         }
28     }
29     return dist;

```

```

30 }

```

3.2 Algoritmo Bellman-Ford

```

1  /*
2  * Bellman-Ford Algorithm
3  * Calcula la distancia m s corta desde un nodo origen, detecta ciclos
4  * negativos.
5  * Complejidad:  $O(V * E)$ 
6  */
7  #include <bits/stdc++.h>
8  using namespace std;
9
10 const int INF = 1e9;
11
12 struct Edge {
13     int u, v, w;
14 };
15
16 vector<int> bellman_ford(int n, int src, const vector<Edge>& edges) {
17     vector<int> dist(n, INF);
18     dist[src] = 0;
19
20     for (int i = 1; i < n; ++i) {
21         for (auto [u, v, w] : edges) {
22             if (dist[u] < INF && dist[v] > dist[u] + w)
23                 dist[v] = dist[u] + w;
24         }
25     }
26
27     // Detecci n de ciclo negativo
28     for (auto [u, v, w] : edges) {
29         if (dist[u] < INF && dist[v] > dist[u] + w)
30             throw runtime_error("Negative_weight_cycle_detected");
31     }
32
33     return dist;
34 }

```

Conteo de componentes conexas

```

1  /*
2  * Conteo de componentes conexas en grafo no dirigido

```

```

3  */
4
5  #include <bits/stdc++.h>
6  using namespace std;
7
8  int count_components(int n, const vector<vector<int>>& adj) {
9      vector<bool> visited(n, false);
10     int count = 0;
11     for (int u = 0; u < n; ++u) {
12         if (!visited[u]) {
13             dfs(u, adj, visited);
14             ++count;
15         }
16     }
17     return count;
18 }

```

3.3 Union Find

```

1  /*
2   * Union-Find (Disjoint Set Union - DSU)
3   * Operaciones eficientes de union y búsqueda de conjuntos disjuntos.
4   * Soporta compresión de caminos y union por tamaño o rango.
5   * Complejidad amortizada: O(α(n)) por operación
6   */
7
8  #include <bits/stdc++.h>
9  using namespace std;
10
11 struct DSU {
12     vector<int> parent, size;
13
14     DSU(int n) {
15         parent.resize(n);
16         size.assign(n, 1);
17         iota(parent.begin(), parent.end(), 0); // parent[i] = i
18     }
19
20     int find(int u) {
21         if (parent[u] != u)
22             parent[u] = find(parent[u]); // compresión de caminos
23         return parent[u];
24     }

```

```

25
26     bool unite(int u, int v) {
27         u = find(u), v = find(v);
28         if (u == v) return false;
29         if (size[u] < size[v]) swap(u, v); // union por tamaño
30         parent[v] = u;
31         size[u] += size[v];
32         return true;
33     }
34
35     bool same(int u, int v) {
36         return find(u) == find(v);
37     }
38 };

```

4 Manejo de Strings

4.1 Sub String

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  int main() {
6      string s = "ProgramacionCompetitiva";
7
8      string sub1 = s.substr(0, 11); // "Programacion"
9      string sub2 = s.substr(11, 11); // "Competitiva"
10     string sub3 = s.substr(5);      // desde el índice 5 hasta el final
11                                     : "amacionCompetitiva"
12
13     cout << sub1 << '\n';
14     cout << sub2 << '\n';
15     cout << sub3 << '\n';
16
17     return 0;
18 }

```

4.2 KMP

```

1  /*
2   * KMP - Knuth-Morris-Pratt
3   * Busca todas las ocurrencias del patrón en el texto

```

```

4  * Complejidad:  $O(n + m)$ , donde  $n$  = tamaño del texto,  $m$  = tamaño del
   patron
5  */
6
7  vector<int> prefix_function(const string &pat) {
8      int m = pat.size();
9      vector<int> pi(m);
10     for (int i = 1; i < m; ++i) {
11         int j = pi[i - 1];
12         while (j > 0 && pat[i] != pat[j])
13             j = pi[j - 1];
14         if (pat[i] == pat[j])
15             ++j;
16         pi[i] = j;
17     }
18     return pi;
19 }
20
21 vector<int> kmp_search(const string &text, const string &pat) {
22     vector<int> pi = prefix_function(pat);
23     vector<int> occ;
24     int n = text.size(), m = pat.size(), j = 0;
25     for (int i = 0; i < n; ++i) {
26         while (j > 0 && text[i] != pat[j])
27             j = pi[j - 1];
28         if (text[i] == pat[j])
29             ++j;
30         if (j == m) {
31             occ.push_back(i - m + 1); // ocurrencia encontrada
32             j = pi[j - 1];
33         }
34     }
35     return occ;
36 }
37
38 /*
39  * Z-Algorithm
40  *  $Z[i]$  = longitud del prefijo común entre  $s$  y  $s[i..]$ 
41  * Sirve para buscar un patrón en texto en  $O(n + m)$ 
42  */
43
44 vector<int> z_function(const string &s) {
45     int n = s.size();

```

```

46     vector<int> z(n);
47     for (int i = 1, l = 0, r = 0; i < n; ++i) {
48         if (i <= r)
49             z[i] = min(r - i + 1, z[i - l]);
50         while (i + z[i] < n && s[z[i]] == s[i + z[i]])
51             ++z[i];
52         if (i + z[i] - 1 > r)
53             l = i, r = i + z[i] - 1;
54     }
55     return z;
56 }
57
58 // Buscar patrón en texto usando Z-function
59 vector<int> z_search(const string &text, const string &pat) {
60     string s = pat + '#' + text;
61     vector<int> z = z_function(s);
62     vector<int> occ;
63     int m = pat.size();
64     for (int i = m + 1; i < (int)s.size(); ++i) {
65         if (z[i] == m)
66             occ.push_back(i - m - 1); // ocurrencia en texto
67     }
68     return occ;
69 }

```

5 Trucos Varios

5.1 Next Permutation

```

1  //Genera todas las permutaciones en Orden lexicografico
2  // La secuencia debe estar previamente ordenada para que funcione
3  vector<int> v = {1, 2, 3};
4  do {for (int x : v) cout << x << '␣';
5      cout << '\n';
6  } while (next_permutation(v.begin(), v.end()));
7  // Salidas 1 2 3 // 1 3 2 // 2 1 3 // 2 3 1 // 3 1 2 // 3 2 1

```

5.2 Uso de Fixed

```

1  #include <iostream>
2  #include <iomanip> // std::fixed, std::setprecision
3  using namespace std;
4

```

```

5 int main() {
6     double pi = 3.1415926535;
7     cout << fixed << setprecision(3) << pi << '\n'; // Output: 3.142
8 }

```

5.3 Equivante fixed con printf

5.4 iota

Para generar arreglos consecutivos

```

1 #include <numeric>
2 vector<int> v(5); // [0, 0, 0, 0, 0]
3 iota(v.begin(), v.end(), 1); // [1, 2, 3, 4, 5]

```

5.5 Conversion de unidades

```

1 /*
2  * Conversiones de unidades
3  * Autor: Jairo Flores
4  * Conversiones comunes para longitud, masa, tiempo, temperatura, etc.
5  */
6
7 #include <bits/stdc++.h>
8 using namespace std;
9
10 // Longitud
11 double km_to_m(double km) { return km * 1000.0; }
12 double m_to_km(double m) { return m / 1000.0; }
13 double m_to_cm(double m) { return m * 100.0; }
14 double cm_to_m(double cm) { return cm / 100.0; }
15 double inch_to_cm(double in) { return in * 2.54; }
16 double cm_to_inch(double cm) { return cm / 2.54; }
17 double ft_to_m(double ft) { return ft * 0.3048; }
18 double m_to_ft(double m) { return m / 0.3048; }
19
20 // Masa
21 double kg_to_g(double kg) { return kg * 1000.0; }
22 double g_to_kg(double g) { return g / 1000.0; }
23 double lb_to_kg(double lb) { return lb * 0.45359237; }
24 double kg_to_lb(double kg) { return kg / 0.45359237; }
25
26 // Tiempo
27 double hr_to_min(double hr) { return hr * 60.0; }

```

```

28 double min_to_sec(double min) { return min * 60.0; }
29 double sec_to_ms(double sec) { return sec * 1000.0; }
30
31 // Temperatura
32 double celsius_to_fahrenheit(double c) { return (c * 9.0 / 5.0) + 32.0; }
33 double fahrenheit_to_celsius(double f) { return (f - 32.0) * 5.0 / 9.0; }
34 double celsius_to_kelvin(double c) { return c + 273.15; }
35 double kelvin_to_celsius(double k) { return k - 273.15; }
36
37 // Velocidad
38 double kmph_to_mps(double kmph) { return kmph / 3.6; }
39 double mps_to_kmph(double mps) { return mps * 3.6; }
40 double mph_to_kmph(double mph) { return mph * 1.60934; }
41 double kmph_to_mph(double kmph) { return kmph / 1.60934; }
42
43 // Area
44 double m2_to_km2(double m2) { return m2 / 1e6; }
45 double km2_to_m2(double km2) { return km2 * 1e6; }
46 double acre_to_m2(double acre) { return acre * 4046.85642; }
47 double m2_to_acre(double m2) { return m2 / 4046.85642; }
48
49 // Volumen
50 double l_to_ml(double l) { return l * 1000.0; }
51 double ml_to_l(double ml) { return ml / 1000.0; }
52 double gal_to_l(double gal) { return gal * 3.78541; } // gal (US)
53 double l_to_gal(double l) { return l / 3.78541; }
54
55 // Binario
56 double kb_to_bytes(double kb) { return kb * 1024.0; }
57 double mb_to_bytes(double mb) { return mb * 1024.0 * 1024.0; }
58 double gb_to_bytes(double gb) { return gb * 1024.0 * 1024.0 * 1024.0; }
59 double bytes_to_kb(double b) { return b / 1024.0; }
60 double bytes_to_mb(double b) { return b / (1024.0 * 1024.0); }
61 double bytes_to_gb(double b) { return b / (1024.0 * 1024.0 * 1024.0); }
62
63 // Prueba
64 int main() {
65     cout << fixed << setprecision(2);
66     cout << "5_km_m=" << km_to_m(5) << "\n";
67     cout << "100_F_C=" << fahrenheit_to_celsius(100) << "\n";
68     cout << "1_acre_m2=" << acre_to_m2(1) << "\n";

```

```

69     cout << "1.5_GB=" << gb_to_bytes(1.5) << "bytes\n";
70     return 0;
71 }

```

5.6 Salidas varias

```

1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      // \n: salto de linea
6      cout << "Hola\nMundo" << endl;
7
8      // \t: tabulacion horizontal
9      cout << "Nombre\tEdad\nJuan\t20\n";
10
11     // \\: imprime el backslash
12     cout << "Ruta: C:\\Users\\Jairo\\Documentos\n";
13
14     // \": imprime comillas dobles
15     cout << "Dijo: \"Hola mundo\"\n";
16
17     // \': imprime comillas simples
18     cout << "Letra: \'A\'\n";
19
20     // \a: beep (no siempre funciona en todas consolas)
21     cout << "\a"; // intento de alerta sonora
22
23     // \b: retroceso (borra el caracter anterior en algunas consolas)
24     cout << "ABC\bD\n"; // imprime ABD (borra la C visualmente)
25
26     // \r: retorno de carro (reinicia al inicio de la linea)
27     cout << "Sobrescribe\rHola\n"; // Holaescribe
28
29     // \f y \v: no tienen mucho efecto visible en terminales modernas
30     cout << "Primera_linea\fSegunda_linea(form_feed)\n";
31     cout << "Primera_linea\vSegunda_linea(vertical_tab)\n";
32
33     // \0: caracter nulo, marca fin de cadena en arreglos de char
34     char palabra[] = {'H', 'o', 'l', 'a', '\0'};
35     cout << "Cadena_terminada_por_nulo: " << palabra << endl;
36
37     return 0;

```

```

38 }

```

5.7 Aritmetica con Strings

```

1  #include <iostream>
2  #include <string>
3  #include <algorithm>
4
5  // Suma de strings que representan n meros enteros positivos
6  std::string sumaStrings(const std::string &a, const std::string &b) {
7      std::string res;
8      int carry = 0;
9      int i = (int)a.size() - 1;
10     int j = (int)b.size() - 1;
11
12     while(i >= 0 || j >= 0 || carry) {
13         int x = i >= 0 ? a[i] - '0' : 0;
14         int y = j >= 0 ? b[j] - '0' : 0;
15         int suma = x + y + carry;
16         carry = suma / 10;
17         res.push_back((suma % 10) + '0');
18         i--; j--;
19     }
20     std::reverse(res.begin(), res.end());
21     return res;
22 }
23
24 // Compara si a >= b (ambos positivos y sin ceros a la izquierda)
25 bool mayorIgual(const std::string &a, const std::string &b) {
26     if (a.size() != b.size()) return a.size() > b.size();
27     return a >= b;
28 }
29
30 // Resta de strings a - b (suponiendo a >= b)
31 std::string restaStrings(const std::string &a, const std::string &b) {
32     if (!mayorIgual(a, b)) return "Error: a < b";
33
34     std::string res;
35     int i = (int)a.size() - 1;
36     int j = (int)b.size() - 1;
37     int borrow = 0;
38
39     while(i >= 0) {

```

```

40     int x = a[i] - '0' - borrow;
41     int y = j >= 0 ? b[j] - '0' : 0;
42
43     if (x < y) {
44         x += 10;
45         borrow = 1;
46     } else {
47         borrow = 0;
48     }
49
50     res.push_back((x - y) + '0');
51     i--; j--;
52 }
53
54 // Quitar ceros a la izquierda
55 while(res.size() > 1 && res.back() == '0') res.pop_back();
56 std::reverse(res.begin(), res.end());
57 return res;
58 }
59
60 // Multiplicacin de strings
61 std::string multiplicacionStrings(const std::string &a, const std::
    string &b) {
62     int n = (int)a.size();
63     int m = (int)b.size();
64     std::string res(n + m, '0');
65
66     for(int i = n - 1; i >= 0; i--) {
67         int carry = 0;
68         int x = a[i] - '0';
69         for(int j = m - 1; j >= 0; j--) {
70             int y = b[j] - '0';
71             int suma = (res[i + j + 1] - '0') + x * y + carry;
72             res[i + j + 1] = (suma % 10) + '0';
73             carry = suma / 10;
74         }
75         res[i] += carry;
76     }
77
78 // Quitar ceros a la izquierda
79 int pos = 0;
80 while (pos < (int)res.size() - 1 && res[pos] == '0') pos++;
81 return res.substr(pos);

```

```

82 }
83
84 // Prueba r pida
85 int main() {
86     std::string A = "123456789123456789";
87     std::string B = "987654321987654321";
88
89     std::cout << "Suma:_" << sumaStrings(A, B) << "\n";
90     std::cout << "Resta_(B_-_A):_" << restaStrings(B, A) << "\n"; // B
    > A
91     std::cout << "Multiplicacion:_" << multiplicacionStrings(A, B) << "\
    n";
92
93     return 0;
94 }

```

5.8 Aritmetica con Strings segun deepseek

```

1  #include <iostream>
2  #include <string>
3  #include <algorithm>
4  using namespace std;
5
6  // Funci n auxiliar para eliminar ceros a la izquierda
7  string removeLeadingZeros(string num) {
8      size_t nonZeroIndex = num.find_first_not_of('0');
9      if (nonZeroIndex == string::npos) {
10         return "0";
11     }
12     return num.substr(nonZeroIndex);
13 }
14
15 // Suma de dos n meros representados como strings
16 string addStrings(string num1, string num2) {
17     num1 = removeLeadingZeros(num1);
18     num2 = removeLeadingZeros(num2);
19
20     int i = num1.length() - 1;
21     int j = num2.length() - 1;
22     int carry = 0;
23     string result;
24
25     while (i >= 0 || j >= 0 || carry > 0) {

```



```

26     int digit1 = (i >= 0) ? num1[i--] - '0' : 0;
27     int digit2 = (j >= 0) ? num2[j--] - '0' : 0;
28     int sum = digit1 + digit2 + carry;
29     carry = sum / 10;
30     result.push_back(sum % 10 + '0');
31 }
32
33 reverse(result.begin(), result.end());
34 return result.empty() ? "0" : result;
35 }
36
37 // Compara dos numeros representados como strings (retorna 1 si num1 >
    num2, -1 si num1 < num2, 0 si iguales)
38 int compareStrings(string num1, string num2) {
39     num1 = removeLeadingZeros(num1);
40     num2 = removeLeadingZeros(num2);
41
42     if (num1.length() > num2.length()) return 1;
43     if (num1.length() < num2.length()) return -1;
44
45     for (int i = 0; i < num1.length(); i++) {
46         if (num1[i] > num2[i]) return 1;
47         if (num1[i] < num2[i]) return -1;
48     }
49
50     return 0;
51 }
52
53 // Resta de dos numeros representados como strings (num1 - num2, asume
    num1 >= num2)
54 string subtractStrings(string num1, string num2) {
55     num1 = removeLeadingZeros(num1);
56     num2 = removeLeadingZeros(num2);
57
58     int i = num1.length() - 1;
59     int j = num2.length() - 1;
60     int borrow = 0;
61     string result;
62
63     while (i >= 0) {
64         int digit1 = num1[i--] - '0' - borrow;
65         int digit2 = (j >= 0) ? num2[j--] - '0' : 0;
66         borrow = 0;

```

```

67
68         if (digit1 < digit2) {
69             digit1 += 10;
70             borrow = 1;
71         }
72
73         result.push_back((digit1 - digit2) + '0');
74     }
75
76     reverse(result.begin(), result.end());
77     return removeLeadingZeros(result);
78 }
79
80 // Multiplicacion de dos numeros representados como strings
81 string multiplyStrings(string num1, string num2) {
82     num1 = removeLeadingZeros(num1);
83     num2 = removeLeadingZeros(num2);
84
85     if (num1 == "0" || num2 == "0") return "0";
86
87     int n1 = num1.length();
88     int n2 = num2.length();
89     string result(n1 + n2, '0');
90
91     for (int i = n1 - 1; i >= 0; i--) {
92         for (int j = n2 - 1; j >= 0; j--) {
93             int product = (num1[i] - '0') * (num2[j] - '0') + (result[i
                + j + 1] - '0');
94             result[i + j + 1] = product % 10 + '0';
95             result[i + j] += product / 10;
96         }
97     }
98
99     return removeLeadingZeros(result);
100 }
101
102 // Division de dos numeros representados como strings (division
    entera)
103 string divideStrings(string num1, string num2) {
104     num1 = removeLeadingZeros(num1);
105     num2 = removeLeadingZeros(num2);
106
107     if (num2 == "0") return "Error: Divisi n por cero";

```

```

108     if (compareStrings(num1, num2) < 0) return "0";
109
110     string quotient;
111     string current;
112
113     for (int i = 0; i < num1.length(); i++) {
114         current += num1[i];
115         current = removeLeadingZeros(current);
116         if (compareStrings(current, num2) < 0) {
117             quotient += "0";
118             continue;
119         }
120
121         int count = 0;
122         string temp = num2;
123         while (compareStrings(current, temp) >= 0) {
124             temp = addStrings(temp, num2);
125             count++;
126         }
127
128         quotient += to_string(count);
129         temp = multiplyStrings(num2, to_string(count));
130         current = subtractStrings(current, temp);
131     }
132
133     quotient = removeLeadingZeros(quotient);
134     return quotient.empty() ? "0" : quotient;
135 }
136
137 int main() {
138     string num1, num2;
139     char op;
140
141     cout << "Ingrese el primer nmero: ";
142     cin >> num1;
143     cout << "Ingrese el segundo nmero: ";
144     cin >> num2;
145     cout << "Ingrese la operaci n (+, -, *, /): ";
146     cin >> op;
147
148     string result;
149     switch(op) {
150         case '+':

```

```

151         result = addStrings(num1, num2);
152         break;
153     case '-':
154         if (compareStrings(num1, num2) < 0) {
155             result = "-" + subtractStrings(num2, num1);
156         } else {
157             result = subtractStrings(num1, num2);
158         }
159         break;
160     case '*':
161         result = multiplyStrings(num1, num2);
162         break;
163     case '/':
164         result = divideStrings(num1, num2);
165         break;
166     default:
167         result = "Operaci n no v lida";
168     }
169
170     cout << "Resultado: " << result << endl;
171     return 0;
172 }

```

5.9 Problema M codigo

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  vector <int> tabla(10000);
4  int __find(int x) {
5      if (tabla[x] != x)
6          tabla[x] = __find(tabla[x]);
7      return tabla[x];
8  }
9  void __union(int a, int b) {
10     a = __find(a);
11     b = __find(b);
12     if (a != b)
13         tabla[b] = a;
14 }
15 int main() {
16     int n;
17     scanf("%d", &n);
18     vector<int> dx(n + 10);

```

```

19     vector<int> dy(n + 10);
20     vector<int> dr(n + 10);
21     for (int i = 0; i < n; i++) {
22         scanf("%d_%d_%d", &dx[i], &dy[i], &dr[i]);
23         tabla[i] = i;
24     }
25     for (int i = 0; i < n; i++) {
26         for (int j = i + 1; j < n; j++) {
27             int hx = dx[i] - dx[j];
28             int hy = dy[i] - dy[j];
29             double tagoras = abs(sqrt(pow(hx, 2) + pow(hy, 2)));
30             if (tagoras <= dr[i] + dr[j]) {
31                 __union(i, j);
32             }
33         }
34     }
35     int ans = 0;
36     for (int u = 0; u < n; u++) {
37         if (__find(u) == u)
38             ans++;
39     }
40     printf("%d\n", ans);
41 }

```

5.10 Matriz Viborita

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main() {
6      int n, m;
7      cout << "Introduce el n mero de filas\n";
8      cin >> n;
9      cout << "Introduce el n mero de columnas\n";
10     cin >> m;
11
12     vector<vector<int>> matriz(n, vector<int>(m));
13     int valor = 1;
14
15     for (int i = 0; i < n; i++) {
16         if (i % 2 == 0) {
17             // Izquierda a derecha

```

```

18         for (int j = 0; j < m; j++) {
19             matriz[i][j] = valor++;
20         }
21     } else {
22         // Derecha a izquierda
23         for (int j = m - 1; j >= 0; j--) {
24             matriz[i][j] = valor++;
25         }
26     }
27 }
28
29 // Imprimir la matriz
30 cout << "\nMatriz_viborita:\n";
31 for (const auto& fila : matriz) {
32     for (int x : fila) {
33         cout << x << "\t";
34     }
35     cout << "\n";
36 }
37
38 return 0;
39 }

```

5.11 Triangulo de Pascal

```

1  #include <iostream>
2  #include <vector>
3  #include <iomanip> // para setw
4  using namespace std;
5
6  // Funci n para calcular combinaciones C(n, k)
7  int combinacion(int n, int k) {
8      if (k == 0 || k == n) return 1;
9      int res = 1;
10     for (int i = 1; i <= k; ++i) {
11         res *= n--;
12         res /= i;
13     }
14     return res;
15 }
16
17 int main() {
18     int filas;

```

```

19     cout << "Introduce el n mero de filas: ";
20     cin >> filas;
21
22     for (int i = 0; i < filas; ++i) {
23         // Imprimir espacios para centrar
24         cout << setw((filas - i) * 2);
25         for (int j = 0; j <= i; ++j) {
26             cout << setw(4) << combinacion(i, j);
27         }
28         cout << endl;
29     }
30
31     return 0;
32 }

```

6 Geometry

6.1 Formulas Basicas

6.2 Convex Hull

```

1  /*
2   * Convex Hull - Algoritmo Graham Scan
3   * Autor: Jairo Flores
4   * Entrada: vector de puntos (x, y)
5   * Salida: vector de puntos que forman el hull convexo en orden ccw
6   */
7
8  #include <bits/stdc++.h>
9  using namespace std;
10
11  struct Point {
12      long long x, y;
13      bool operator<(const Point& p) const {
14          return y < p.y || (y == p.y && x < p.x);
15      }
16  };
17
18  // Producto cruzado (vectorial) de AB x AC
19  long long cross(const Point &O, const Point &A, const Point &B) {
20      return (A.x - O.x)*(B.y - O.y) - (A.y - O.y)*(B.x - O.x);
21  }
22

```

```

23 // Funcion que calcula el convex hull y devuelve los vertices en orden
    CCW
24 vector<Point> convexHull(vector<Point> pts) {
25     int n = (int)pts.size();
26     if (n <= 1) return pts;
27
28     // Paso 1: encontrar punto base (con menor y, si empate menor x)
29     sort(pts.begin(), pts.end());
30     Point base = pts[0];
31
32     // Paso 2: ordenar por angulo polar respecto a base
33     sort(pts.begin() + 1, pts.end(), [&](const Point &a, const Point &b)
34         {
35             long long c = cross(base, a, b);
36             if (c == 0) // si son colineales, el mas cercano va primero
37                 return (a.x - base.x)*(a.x - base.x) + (a.y - base.y)*(a.y -
38                     base.y) <
39                     (b.x - base.x)*(b.x - base.x) + (b.y - base.y)*(b.y -
40                         base.y);
41             return c > 0;
42         });
43
44     // Paso 3: construir el hull usando stack
45     vector<Point> hull;
46     for (auto &p : pts) {
47         while ((int)hull.size() > 1 && cross(hull[hull.size()-2], hull.
48             back(), p) <= 0)
49             hull.pop_back();
50         hull.push_back(p);
51     }
52
53     return hull;
54 }
55
56 // Ejemplo de uso
57 int main() {
58     ios::sync_with_stdio(false);
59     cin.tie(nullptr);
60
61     int n; cin >> n;
62     vector<Point> pts(n);
63     for (int i = 0; i < n; i++)
64         cin >> pts[i].x >> pts[i].y;

```

```

61     vector<Point> hull = convexHull(pts);
62
63
64     cout << (int)hull.size() << "\n";
65     for (auto &p : hull)
66         cout << p.x << " " << p.y << "\n";
67
68     return 0;
69 }

```

6.3 Trigonometria

```

1  /*
2   * Trigonometria basica para Programacion Competitiva
3   * Autor: Jairo Flores
4   */
5
6  #include <bits/stdc++.h>
7  using namespace std;
8
9  const double PI = acos(-1);
10
11  // Funciones trigonomtricas
12  // ngulos en radianes (usar grados * PI/180 para convertir)
13
14  // Convertir grados a radianes
15  double deg_to_rad(double deg) {
16      return deg * PI / 180.0;
17  }
18
19  // Convertir radianes a grados
20  double rad_to_deg(double rad) {
21      return rad * 180.0 / PI;
22  }
23
24  // Calcular hipotenusa con pitagoras
25  double hipotenusa(double a, double b) {
26      return sqrt(a*a + b*b);
27  }
28
29  // Ley de senos: a/sin(A) = b/sin(B) = c/sin(C)
30  double ley_de_senos(double a, double A_rad, double B_rad) {
31      // Por ejemplo, dado lado a y angulo A, calcula lado b dado angulo B

```

```

32     return a * sin(B_rad) / sin(A_rad);
33 }
34
35 // Ley de cosenos: c^2 = a^2 + b^2 - 2ab cos(C)
36 double ley_de_cosenos(double a, double b, double C_rad) {
37     return sqrt(a*a + b*b - 2*a*b*cos(C_rad));
38 }
39
40 // Area de un triangulo con dos lados y angulo entre ellos: (1/2)*ab*sin(C)
41 double area_triangulo_trig(double a, double b, double C_rad) {
42     return 0.5 * a * b * sin(C_rad);
43 }
44
45 // Calculo de angulos (ejemplo)
46 double angulo_entre_vectores(double x1, double y1, double x2, double y2)
47 {
48     // Producto punto = |v1||v2|cos(theta)
49     double dot = x1*x2 + y1*y2;
50     double mag1 = sqrt(x1*x1 + y1*y1);
51     double mag2 = sqrt(x2*x2 + y2*y2);
52     return acos(dot / (mag1*mag2)); // devuelve radianes
53 }
54
55 // Distancia entre dos puntos (x1,y1) y (x2,y2)
56 double distancia(double x1, double y1, double x2, double y2) {
57     return sqrt((x1 - x2)*(x1 - x2) + (y1 - y2)*(y1 - y2));
58 }
59
60 // Ejemplo de uso:
61 int main() {
62     ios::sync_with_stdio(false);
63     cin.tie(nullptr);
64
65     double A_deg = 30.0, B_deg = 45.0;
66     double a = 5.0; // lado a
67     double A = deg_to_rad(A_deg);
68     double B = deg_to_rad(B_deg);
69
70     cout << fixed << setprecision(4);
71     cout << "Lado_b_usando_ley_de_senos: " << ley_de_senos(a, A, B) << "

```

```

72 double c = ley_de_cosenos(a, ley_de_senos(a, A, B), deg_to_rad(60));
73 cout << "Lado c usando ley de cosenos: " << c << "\n";
74
75 double area = area_triangulo_trig(a, ley_de_senos(a, A, B),
76     deg_to_rad(60));
77 cout << "Area triangulo con lados y angulo: " << area << "\n";
78
79 double angulo = angulo_entre_vectores(1, 0, 0, 1);
80 cout << "Angulo entre vectores (1,0) y (0,1): " << rad_to_deg(angulo
81     ) << "\n";
82
83 return 0;
84 }

```

7 Math

7.1 Factores de un numero

```

1 vector<ll> all_factors(ll n) {
2     vector<ll> fac = {1};
3     while (n > 1) {
4         const ll p = mpf[n];
5         vector<ll> cur = {1};
6         while (n % p == 0) {
7             n /= p;
8             cur.push_back(cur.back() * p);
9         }
10        vector<ll> tmp;
11        for (auto x : fac)
12            for (auto y : cur) tmp.push_back(x * y);
13        tmp.swap(fac);
14    }
15    return fac;
16 }

```

7.2 Linear Sieve

```

1 constexpr ll MAXN = 1000000;
2 bitset<MAXN> is_prime;
3 vector<ll> primes;
4 ll mpf[MAXN], phi[MAXN], mu[MAXN];
5
6 void sieve() {

```

```

7     is_prime.set();
8     is_prime[1] = 0;
9     mu[1] = phi[1] = 1;
10    for (ll i = 2; i < MAXN; i++) {
11        if (is_prime[i]) {
12            mpf[i] = i;
13            primes.push_back(i);
14            phi[i] = i - 1;
15            mu[i] = -1;
16        }
17        for (ll p : primes) {
18            if (p > mpf[i] || i * p >= MAXN) break;
19            is_prime[i * p] = 0;
20            mpf[i * p] = p;
21            mu[i * p] = -mu[i];
22            if (i % p == 0)
23                phi[i * p] = phi[i] * p, mu[i * p] = 0;
24            else phi[i * p] = phi[i] * (p - 1);
25        }
26    }

```

7.3 Cribita normal

```

1 constexpr int MAXN = 1000000;
2 bitset<MAXN + 1> is_prime;
3
4 void sieve() {
5     is_prime.set(); // Marca todos como potencialmente primos
6     is_prime[0] = is_prime[1] = 0; // 0 y 1 no son primos
7
8     for (int i = 2; i * i <= MAXN; ++i) {
9         if (is_prime[i]) {
10            for (int j = i * i; j <= MAXN; j += i)
11                is_prime[j] = 0; // Marca multiples de i como no
12                                // primos
13        }
14    }

```

7.4 Criba vasito

```

1 int cr[MAXN]; // -1 if prime, some not trivial divisor if not
2 void init_sieve(){
3     memset(cr, -1, sizeof(cr));

```

```

4 fore(i,2,MAXN)if(cr[i]<0)for(11 j=1LL*i*i;j<MAXN;j+=i)cr[j]=i;
5 }
6 map<int,int> fact(int n){ // must call init_cribe before
7 map<int,int> r;
8 while(cr[n]>=0)r[cr[n]]++,n/=cr[n];
9 if(n>1)r[n]++;
10 return r;
11 }

```

7.5 Contar Divisores

```

1 int count_divisors(int n) {
2     int res = 1;
3     for (int i = 2; i * i <= n; ++i) {
4         int count = 0;
5         while (n % i == 0) {
6             n /= i;
7             count++;
8         }
9         res *= (count + 1);
10    }
11    if (n > 1) res *= 2; // n es primo > sqrt(original n)
12    return res;
13 }

```

7.6 Operaciones con matrices

```

1 #include <iostream>
2 #include <vector>
3 #include <cmath>
4 #include <iomanip>
5
6 using namespace std;
7
8 using Matrix = vector<vector<double>>;
9
10 // Funci n para imprimir matriz
11 void printMatrix(const Matrix &mat) {
12     for (const auto &row : mat) {
13         for (double val : row) cout << setw(10) << val << " ";
14         cout << "\n";
15     }
16 }
17

```

```

18 // Intercambia filas r1 y r2 en la matriz
19 void swapRows(Matrix &mat, int r1, int r2) {
20     std::swap(mat[r1], mat[r2]);
21 }
22
23 // Calcula el determinante con eliminaci n Gaussiana
24 double determinant(Matrix mat) {
25     int n = (int)mat.size();
26     double det = 1.0;
27
28     for (int i = 0; i < n; i++) {
29         // Buscar fila con pivote no cero
30         int pivot = i;
31         while (pivot < n && fabs(mat[pivot][i]) < 1e-12) pivot++;
32         if (pivot == n) return 0; // matriz singular
33
34         if (pivot != i) {
35             swapRows(mat, i, pivot);
36             det = -det; // cambiar signo si intercambiamos filas
37         }
38
39         det *= mat[i][i];
40         if (fabs(mat[i][i]) < 1e-12) return 0;
41
42         // Eliminar debajo del pivote
43         for (int r = i + 1; r < n; r++) {
44             double factor = mat[r][i] / mat[i][i];
45             for (int c = i; c < n; c++) {
46                 mat[r][c] -= factor * mat[i][c];
47             }
48         }
49     }
50
51     return det;
52 }
53
54 // Calcula la inversa usando Gauss-Jordan
55 bool inverse(const Matrix &input, Matrix &inv) {
56     int n = (int)input.size();
57     Matrix mat = input;
58     inv.assign(n, vector<double>(n, 0));
59
60     // Crear matriz identidad en inv

```

```

61     for (int i = 0; i < n; i++) inv[i][i] = 1.0;
62
63     for (int i = 0; i < n; i++) {
64         // Encontrar pivote
65         int pivot = i;
66         for (int r = i + 1; r < n; r++) {
67             if (fabs(mat[r][i]) > fabs(mat[pivot][i])) pivot = r;
68         }
69
70         if (fabs(mat[pivot][i]) < 1e-12) return false; // matriz
            singular
71
72         if (pivot != i) {
73             swapRows(mat, i, pivot);
74             swapRows(inv, i, pivot);
75         }
76
77         double pivot_val = mat[i][i];
78         for (int c = 0; c < n; c++) {
79             mat[i][c] /= pivot_val;
80             inv[i][c] /= pivot_val;
81         }
82
83         // Eliminar otros elementos en columna i
84         for (int r = 0; r < n; r++) {
85             if (r != i) {
86                 double factor = mat[r][i];
87                 for (int c = 0; c < n; c++) {
88                     mat[r][c] -= factor * mat[i][c];
89                     inv[r][c] -= factor * inv[i][c];
90                 }
91             }
92         }
93     }
94
95     return true;
96 }
97
98 // Ejemplo de uso
99 int main() {
100     Matrix arr = {
101         {4, 7, 2},
102         {3, 6, 1},

```

```

103         {2, 5, 1}
104     };
105
106     cout << "Matriz original:\n";
107     printMatrix(arr);
108
109     double det = determinant(arr);
110     cout << "\nDeterminante: " << det << "\n";
111
112     if (fabs(det) < 1e-12) {
113         cout << "La matriz no tiene inversa (det=0)\n";
114     } else {
115         Matrix inv;
116         if (inverse(arr, inv)) {
117             cout << "\nMatriz inversa:\n";
118             printMatrix(inv);
119         } else {
120             cout << "No se pudo calcular la inversa\n";
121         }
122     }
123
124     return 0;
125 }

```

7.7 Fast Exponentiation

```

1 def fastExponentiation(base, exponent):
2     result = 1
3     base = base % MOD
4     while exponent > 0:
5         if (exponent % 2) == 1:
6             result = (result * base) % MOD
7             exponent = exponent // 2
8             base = (base * base) % MOD
9     return result

```